



IL2234

Digital Systems Design and Verification using
Hardware Description Languages

Project Milestone 4

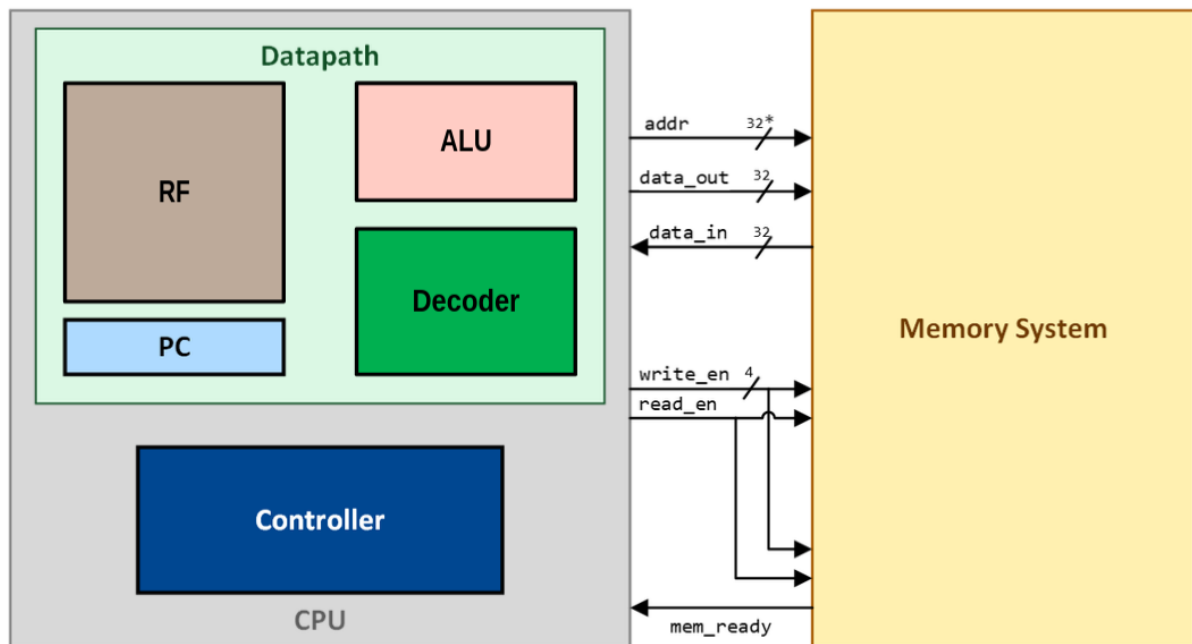
Connecting CPU to Memory and FPGA
Implementation

Part 1: Connecting CPU to Memory

Introduction

After having finished the processor design, you are now able to assemble the complete CPU with the memory system and execute code on it.

System Overview



In this task, you will create a top module containing only a RISC-V processor you designed throughout the previous milestones, and the memory system you designed in Milestone 2. The connections between the two are illustrated in the figure above and must respect the memory behaviour of Milestone 2.

Inside the memory

You can manually write machine code by following the instruction formats described in Milestone 4. This can be useful for testing very simple programs and for debugging.

For more complex codes, we will use a C compiler. Since the CPU follows the RISC-V RV32I instruction set, we can use the corresponding GCC toolchain to compile code for it. For that you will need to install the corresponding packages to be able to use GCC with the RISC-V target, the package name is usually "riscv32-unknown-elf-" but it might vary depending on your specific OS.

The repository to install the toolchain can be found here: <https://github.com/riscv-collab/riscv-gnu-toolchain>

Your KTH Github repository includes the code for examination and the necessary files to compile it. For those of you who may not be able to install the toolchain, the memory file (firmware.coe) for testing and examination is also provided.

The overview of the steps is as follows

1. Build the assembly code using “make all”. Make sure you compile your program using the `-O0` to disable all optimizations.
2. Observe the ELF file by running “`riscv32-unknown-elf-objdump -d firmware.elf`”
3. Generating the SRAM initialisation file (COE) from the `firmware.elf` by running “`./elf2coe.sh`”

Note: This was tested on Ubuntu. Make the adjustments depending on your OS if you are unable to follow these steps.

This is an example of the ELF file contents:

```
firmware.elf:      file format elf32-littleriscv
```

Disassembly of section `.text`:

```
00000000 <_start>:
```

```

0:      00010117      auipc      sp,0x10
4:      00010113      mv        sp,sp
8:      00008517      auipc      a0,0x8
c:      ff850513      addi      a0,a0,-8 # 8000 <counter>
10:     00008597      auipc      a1,0x8
14:     ff058593      addi      a1,a1,-16 # 8000 <counter>
18:     0cc00613      li        a2,204
1c:     00b50c63      beq       a0,a1,34 <_start+0x34>
20:     00062283      lw        t0,0(a2)
24:     00552023      sw        t0,0(a0)
28:     00450513      addi      a0,a0,4
2c:     00460613      addi      a2,a2,4
30:     fedff06f      j         1c <_start+0x1c>
34:     00008517      auipc      a0,0x8
```

```

38:      fd050513      addi      a0,a0,-48 # 8004 <_ebss>
3c:      00008597      auipc     a1,0x8
40:      fc858593      addi      a1,a1,-56 # 8004 <_ebss>
44:      00000293      li        t0,0
48:      00b50863      beq       a0,a1,58 <_start+0x58>
4c:      00552023      sw        t0,0(a0)
50:      00450513      addi      a0,a0,4
54:      ff5ff06f      j         48 <_start+0x48>
58:      008000ef      jal       60 <main>
5c:      0000006f      j         5c <_start+0x5c>

```

00000060 <main>:

```

60:      fe010113      addi      sp,sp,-32 # ffe0 <_ebss+0x7fdc>
64:      00112e23      sw        ra,28(sp)
68:      00812c23      sw        s0,24(sp)
6c:      02010413      addi      s0,sp,32
70:      0000a7b7      lui       a5,0xa
74:      fef42423      sw        a5,-24(s0)
78:      fe042623      sw        zero,-20(s0)
7c:      0280006f      j         a4 <main+0x44>
80:      000087b7      lui       a5,0x8
84:      0007a703      lw        a4,0(a5) # 8000 <counter>
88:      fec42783      lw        a5,-20(s0)
8c:      00f70733      add       a4,a4,a5
90:      000087b7      lui       a5,0x8
94:      00e7a023      sw        a4,0(a5) # 8000 <counter>
98:      fec42783      lw        a5,-20(s0)
9c:      00178793      addi      a5,a5,1

```

a0:	fef42623	sw	a5, -20(s0)
a4:	fec42703	lw	a4, -20(s0)
a8:	06300793	li	a5, 99
ac:	fce7dae3	bge	a5, a4, 80 <main+0x20>
b0:	000087b7	lui	a5, 0x8
b4:	0007a783	lw	a5, 0(a5) # 8000 <counter>
b8:	00078713	mv	a4, a5
bc:	fe842783	lw	a5, -24(s0)
c0:	00e7a023	sw	a4, 0(a5)
c4:	00000013	nop	
c8:	ffdff06f	j	c4 <main+0x64>

Tasks

1. Complete the RTL system for connecting the CPU and memory system.
2. Create a testbench to test the processor with the memory. One of test cases must be the provided program, starting by dump the program on to the memory, releasing the reset to CPU, and waiting until the processor completes the program.

Deliverables

1. Submit the RTL design of your complete system and its testbench(es) on your KTH Github repository in [riscv/top/](#)

Submission date: *before the examination*

Individual submission: *same content is allowed within a group*

Part 2: FPGA Implementation

Introduction

In this task, we will work on the peripheral circuits around the processor and implement the complete system on to the FPGA.

Requirements

The signals between the processor with memory system must be connected to the Urbana board displays, switches, and LEDs as follows:

- The reset signal must be connected to one of the Switches
- CPU clock and reset signal must be connected to the LEDs
- Strobe signals (read_en, write_en and mem_ready) must be connected to the LEDs
- The lowest 16 bits of the *latest written memory data* must be connected to four 7-Segment Displays of the FPGA and the program counter must be connected to the other four 7-segment displays
- The Program must finish in approximately a minute and must be observable, use an appropriate clock frequency to achieve this.

You may reuse the resources you obtained during the lab sessions.

Tasks

1. Complete the RTL system with the additional board-level requirements.
2. Synthesize and implement the system using Vivado, and flash the generated bitstream to the FPGA. Verify that the implementation works functionally well and stable before examination.

Deliverables

1. Submit the RTL design of your complete system on your KTH Github repository in [fpga/m4/](#)
2. [optional] if you have another program of your choice to demonstrate, include the C code and its generated COE with which you tested your CPU inside [firmware/](#)

Submission date: *before the examination*

Individual submission: *same content is allowed within a group*

Final presentation (Individual)

You need to show that the CPU is working by showing the complete execution of the program we provided inside the repository (and/or the program of your choice). Make sure that the program counter and the last valid written data from the memory bus is visible on the 7-Segments and that the program finishes in approximately one minute.



Submit your schematics and drawings that you have created during the milestones, as well as the code that you have developed during the project. We will ask questions on both the schematics and the code, particularly on linking the two by showing where a specific schematic element/connection appears in the code and vice versa.